

Préparation à la revue de code

Explication fonction par fonction, choix techniques, pipelines, schéma de données, benchmarks & tarifs, et questions probables du jury

Document généré le	23 juillet 2026
Branche courante	feature/visio_to_database
Périmètre	6 branches analysées : feature/visio_to_database, feature/dashboard, feature/link_dictaphone_to_database, lab/test_vexa_meeting_baas, lab/benchmark_meeting_bas, features/join-auto-bot-calendars-api, lab/benchmark-STT, feature/dictaphone (v1), feature/speech_to_text_agent
Objectif	Pouvoir défendre chaque choix technique (STT, LLM, DB, architecture bot de visio) devant le professeur, avec chiffres sourcés (benchmarks WER, tarifs) et réponses préparées aux points faibles honnêtement identifiés.

Sommaire

1. Vue d'ensemble de l'architecture & schéma de données global
2. Pipeline visio → base de données **PRIORITAIRE** (branche feature/visio_to_database)
3. Pipeline dictaphone / STT Voxtral (branche feature/dashboard)
4. Liaison dictaphone ↔ base de données (branche feature/link_dictaphone_to_database)
5. Vexa vs Meeting BaaS & connexion calendrier automatique
6. Historique des benchmarks STT (pourquoi Voxtral au final)
7. Benchmarks & tarifs sourcés (Voxtral, Meeting BaaS, Gladia, Vexa)
8. FAQ transversale — questions probables du jury
9. Fiche "points faibles à assumer honnêtement"

2. Pipeline visio → base de données

branche courante

Branche `feature/visio_to_database`. Historique complet des 15 commits : la branche a démarré comme un lab de test de **Vexa** (bot open-source), a ajouté la diarisation, corrigé des bugs de diarisation, pivoté vers **Meeting Baas** comme solution de production (commit `00f2738` "ajout du bot Teams"), simplifié le pipeline, puis affiné `models.py` jusqu'au dernier commit `5da8731`.

```
0dafdaa Initial commit
20bd395 lab/test_vexa_meeting_baas: initial commit
08b4416 feat: add speaker diarization
dab9867 bug/fix diarisation
d717cf8 changement en audio only mais pas réussi à supprimer les captures d'écran
e04dccf modification de vexa bot
ab2aaae ajout du push vers la base
5792b63 simplification de stt à base de données
30ebc14 refacto du projet et même technique que dictaphone to database
00f2738 ajout du bot Teams pour visio_to_database
db3e173 ajout des fichiers plus standardisation de la branche par rapport aux autres
7cf38ac modification et simplification de certaines fonctions
b09cb05 update def generate_report (suppression docstring)
5da8731 modification de models.py (HEAD)
```

backend/config.py — centralisation des clés d'API

`get_api_key(env_var)` → lit `os.environ`, lève une `EnvironmentError` explicite si absente (message renvoyant vers `.env.example`). Fail-fast plutôt qu'un `KeyError` obscur plus loin dans l'exécution.

`get_vexa_api_key / get_meeting_baas_api_key / get_gladia_api_key / get_together_api_key / get_database_url` → wrappers typés, un par service externe.

Pourquoi : un seul point de vérité pour les secrets, aucune clé hardcodée dans le code (bonne pratique sécurité basique mais souvent oubliée).

backend/meetingbaas_client.py — client HTTP Meeting Baas

Constantes : `API_BASE_URL="https://api.meetingbaas.com/v2"`, `POLL_INTERVAL_SECONDS=15`, `POLL_TIMEOUT_SECONDS=3600`, `EXTRA_WAIT_AFTER_END_SECONDS=300`, `DEFAULT_TRANSCRIPTION_PROVIDER="gladia"`, `DEFAULT_RECORDING_MODE="audio_only"`.

`create_bot(api_key, meeting_url, bot_name, transcription_enabled, provider, recording_mode)` → `POST /v2/bots`. Envoie le bot dans la réunion : URL, nom, mode d'enregistrement, timeout salle d'attente 600s, config de transcription si activée. Retourne le `bot_id`.

`get_bot_status(api_key, bot_id)` → `GET /v2/bots/{id}`, statut courant + lien média si disponible.

`list_bots_history(api_key, limit)` → `GET /v2/bots`, avec désimbrication défensive du JSON (plusieurs formats de réponse possibles côté API).

remove_bot(api_key, bot_id) → DELETE /v2/bots/{id}, best-effort (catch large, ne fait jamais planter l'appelant).

wait_for_completion(api_key, bot_id) → polling bloquant toutes les 15s pendant 1h max. Si failed/error → exception. Si call_ended/completed + média présent → retour immédiat, sinon jusqu'à 5 min de grâce supplémentaires (le média met du temps à être finalisé côté Meeting BaaS) avant d'abandonner en retournant le dernier statut connu.

Pourquoi Meeting BaaS : les protocoles de visio (Teams surtout) sont propriétaires et complexes à automatiser soi-même (join headless, salle d'attente, permissions). Déléguer à un service spécialisé multi-plateforme = choix *build vs buy* classique, justifié par le temps de projet contraint.

Fragile : polling synchrone bloquant avec requests (pas httpx.AsyncClient) dans une BackgroundTask FastAPI → consomme un thread jusqu'à 1h, ne scale pas avec beaucoup de réunions simultanées. Pas de retry/backoff sur erreurs réseau transitoires. Timeout fixe non configurable par env. Une architecture event-driven (webhook Meeting BaaS, comme dans la branche calendrier) serait plus propre qu'un polling.

backend/meetingbaas_teams_bot.py — CLI de test

Script argparse exposant manuellement toutes les fonctions du client (envoyer un bot, statut, historique, transcript, déclencher save_meeting) — outil de test manuel indépendant de l'API FastAPI, preuve de tests avant intégration.

backend/transcript.py — normalisation de la réponse Meeting BaaS

load_transcription_if_needed(payload) → le champ transcription est parfois une URL S3 présignée (fichier volumineux) plutôt que le JSON inline ; télécharge et remplace. Idempotent.

extract_diarized_transcript(payload) → lit transcription.result.utterances (format Gladia : un tour de parole = speaker/début/fin/texte), normalise les labels vides en "Inconnu".

group_by_speaker(segments) → regroupe le texte par intervenant.

build_transcript_text(segments) → construit le texte "Speaker: texte" ligne par ligne — **c'est ce texte qui est envoyé au LLM.**

build_speakers_list(segments) → liste des speakers au format DB à partir des labels de diarisation.

build_speakers_from_participants(participants) → **fallback** si la diarisation échoue : reconstruit les speakers depuis la vraie liste de participants Meeting BaaS (noms réels).

build_meeting_name(participants, bot_name, fallback_name) → nomme le recap avec la liste des participants humains, en excluant le bot (comparaison par nom exact).

Pourquoi séparer ce fichier de save_meeting.py : fonctions pures (mêmes entrées → mêmes sorties, testables sans mock DB/API) vs orchestration à effets de bord — plus facile à tester unitairement.

Fragile : `build_meeting_name` exclut le bot par égalité de chaîne exacte sur le nom — cassant si Meeting BaaS renvoie un nom légèrement différent (espace, casse). Un flag `is_bot` fourni par l'API serait plus robuste.

backend/llm.py — génération du compte-rendu

`generate_report(transcript)` → client Together AI, modèle configurable (`TOGETHER_MODEL`, défaut `Qwen/Qwen3.7-Max`), prompt système chargé depuis `agent_context.txt`, `temperature=0`, `response_format={"type":"json_object"}`, consomme le stream puis `json.loads` le résultat.

`agent_context.txt` impose au LLM de répondre uniquement avec `{summary, themes, actions}` — **sans le champ `speakers`**, ajouté programmatiquement après coup dans `save_meeting.py`.

Pourquoi le LLM ne génère jamais les `speakers` : fiabilité — les intervenants viennent toujours de données réelles (diarisation Gladia ou participants Meeting BaaS), jamais d'une hallucination potentielle du modèle.
Pourquoi `temperature=0` + `JSON mode` : un compte-rendu doit être factuel et reproductible, pas créatif ; le `JSON mode` fiabilise le parsing en aval.

Fragile : aucun `try/except` autour de `json.loads()` ni validation Pydantic du schéma retourné — une réponse LLM malformée fait planter tout `save_meeting`.

backend/save_meeting.py — orchestration métier

`save_meeting(bot_result, bot_name)` → `{id, created_at}` — point de convergence du pipeline :

1. `load_transcription_if_needed` + `extract_diarized_transcript`
2. Si segments diarisés OK → `build_transcript_text` + `build_speakers_list`
3. Sinon (diarisation ratée) → fallback texte brut + `build_speakers_from_participants` (le pipeline ne s'arrête jamais totalement)
4. `build_meeting_name`
5. `generate_report(transcript_text)` puis ajout de `report["speakers"]`
6. Construction d'un Recap SQLAlchemy (`source="visio"`), `add/commit/refresh`, fermeture systématique en `finally`

Pourquoi `source="visio"` vs `"dictaphone"` : les deux pipelines convergent vers la même table recaps avec un simple discriminant — le frontend affiche les deux types de comptes-rendus de façon unifiée sans dupliquer le modèle.

backend/models.py & db.py

Voir le schéma détaillé en section 1. `db.py` : engine SQLAlchemy Postgres (`psycopg2-binary`), `SessionLocal` (`autoflush=False`), Base déclarative — configuration minimaliste standard.

backend/main.py — API FastAPI

`POST /bots` → reçoit `{meeting_url, bot_name, provider, recording_mode}`, lance `run_meeting_bot` en `BackgroundTasks` (réponse HTTP immédiate `{"status":"ok"}` sans attendre la fin de la réunion, évite un timeout HTTP côté frontend puisque l'attente peut durer jusqu'à 1h).

run_meeting_bot → crée le bot, attend sa complétion, appelle save_meeting. En cas d'exception à n'importe quelle étape → log + remove_bot (évite un bot orphelin).

GET /bots/{id} → proxy vers get_bot_status, toute exception → HTTPException 502 (erreur passerelle amont, cohérent puisque c'est un service tiers qui échoue).

GET /bots · DELETE /bots/{id} → historique / retrait manuel.

Pourquoi BackgroundTasks plutôt que Celery/RQ : simplicité pour un MVP étudiant, pas d'infra supplémentaire (broker Redis) à déployer.

Fragile : si le process FastAPI redémarre pendant qu'un bot attend la fin de sa réunion, la tâche de fond est perdue silencieusement — pas de persistance de l'état "en cours".

backend/vexa_bot.py — code résiduel (non utilisé)

Client pour l'API Vexa (<https://api.cloud.vexa.ai>), alternative open-source. parse_meeting détecte la plateforme (regex Google Meet, ou extraction Conference ID/Passcode Teams). create_bot, get_transcript, poll_transcript suivent le même schéma que le client Meeting BaaS. **Ce fichier n'est appelé nulle part dans main.py** — vestige du POC initial avant le pivot vers Meeting BaaS (voir section 5).

Le pipeline complet, étape par étape

1. Frontend → POST /bots {meeting_url, bot_name, provider, recording_mode}
2. FastAPI répond "ok" immédiatement, lance run_meeting_bot en tâche de fond
3. create_bot() → POST api.meetingbaas.com/v2/bots
→ Meeting BaaS rejoint LUI-MÊME la réunion Teams (protocole propriétaire, pas de code custom)
→ Meeting BaaS enregistre l'audio (mode "audio_only" par défaut)
→ Meeting BaaS envoie l'audio à GLADIA (provider configuré) pour transcription+diarisation
(donc PAS Voxtral ici, contrairement au dictaphone – Voxtral n'intervient jamais côté visio)
4. wait_for_completion() → polling 15s jusqu'à 1h, jusqu'à média finalisé
5. Statut final : transcription = JSON inline OU URL S3 présignée
6. save_meeting(bot_result, bot_name) :
 - a. load_transcription_if_needed → télécharge si URL S3
 - b. extract_diarized_transcript → segments {speaker, start, end, text} (format Gladia "utterances")
 - c. build_transcript_text → texte "Speaker: phrase"
 - d. generate_report() → appel Together AI/Qwen → {summary, themes, actions}
 - e. report["speakers"] = speakers réels (jamais générés par le LLM)
 - f. Recap(source="visio", ...) → INSERT Postgres (SQLAlchemy)
7. Erreur à une étape quelconque → remove_bot() (nettoyage) + log serveur

Questions probables du jury — pipeline visio

Q. Pourquoi Meeting BaaS plutôt que coder l'intégration Teams/Zoom/Meet soi-même ?

R. Les protocoles de visio propriétaires sont complexes à automatiser (headless join, salle d'attente, permissions). Meeting BaaS gère plusieurs plateformes avec une seule API REST — build vs buy, gain de temps énorme pour un projet étudiant avec deadline.

Q. Pourquoi Gladia comme provider de transcription côté visio et pas Voxtral partout ?

R. Gladia est intégré nativement à Meeting BaaS (`transcription_config.provider`) — pas besoin d'envoyer l'audio à un autre service, Meeting BaaS orchestre l'appel en interne. Voxtral (Mistral) est utilisé séparément pour le dictaphone, où le backend reçoit un fichier brut et doit lui-même appeler un service STT.

Q. Que se passe-t-il si `wait_for_completion timeout` après 1h ?

R. Une `TimeoutError` est levée, catchée dans `run_meeting_bot`, qui logue l'erreur et retire le bot via `remove_bot`. Aucun recap n'est enregistré dans ce cas.

Q. Pourquoi `BackgroundTasks` FastAPI plutôt qu'une vraie queue (Celery/RQ) ?

R. Simplicité pour le MVP, pas d'infra supplémentaire à déployer. Limite assumée : perte silencieuse de la tâche si le serveur redémarre pendant l'attente — acceptable pour une démo, à améliorer en prod.

Q. Quel est le lien entre Recap et un User dans le modèle actuel ?

R. Aucun actuellement — le dernier commit a retiré `user_id` du modèle Recap côté visio. C'est un chantier en cours : l'authentification n'est pas encore branchée sur ce flux (contrairement au dictaphone qui a un `user_id` nullable). À assumer plutôt qu'à nier.

Q. Pourquoi reporting est stocké en JSONB plutôt qu'en colonnes séparées ?

R. Le format généré par le LLM peut évoluer (nouveaux champs) sans migration SQL à chaque itération de prompt ; Postgres JSONB reste indexable si besoin plus tard.

Q. Comment la diarisation est-elle obtenue ?

R. Gladia (via Meeting BaaS) renvoie `result.utterances`, chaque utterance ayant `speaker/start/end/text`. `extract_diarized_transcript` normalise ce format en segments exploitables.

Q. Que se passe-t-il si la diarisation échoue ou renvoie 0 segment ?

R. `save_meeting` bascule sur un chemin de secours : texte brut de transcription s'il existe, et speakers reconstruits depuis la vraie liste de participants Meeting BaaS plutôt que de planter.

Q. Pourquoi `vexa_bot.py` est encore dans le repo s'il n'est jamais appelé ?

R. Vestige du POC initial (`lab/test_vexa_meeting_baas`) qui comparait Vexa à Meeting BaaS avant de trancher. Gardé comme trace de la démarche de choix technique — à nettoyer avant un merge en prod. Réponse honnête plutôt que de prétendre qu'il sert encore.

Q. Pourquoi séparer `transcript.py` (parsing) de `save_meeting.py` (orchestration) ?

R. Séparation des responsabilités : fonctions pures testables sans mock (transcript.py) vs effets de bord réseau/DB concentrés (save_meeting.py) — plus facile à tester et faire évoluer indépendamment.

3. Pipeline dictaphone / STT Voxtral

[feature/dashboard](#)

Structure : `main.py` (API FastAPI), `stt.py` (transcription Voxtral), `llm.py` (compte-rendu), `agent_context.txt` (prompt système). `db.py/models.py` présents mais **vides (0 octet)** sur cette branche — la persistance DB est branchée plus tard sur `feature/link_dictaphone_to_database` (section 4). Dernier commit : `bf35320` "ajout du paramètre de la température à 0".

backend/stt.py

`convert_to_wav(input_path)` → conversion ffmpeg vers wav 16kHz mono. **△ définie deux fois dans le fichier – la seconde écrase la première (dead code, doublon d'itération).**

`callapi(file_path)` → encode le wav en base64, appelle `client.chat.complete()` de Mistral avec le modèle configurable `MISTRAL_MODEL` (défaut `voxtral-small-latest`), `temperature=0.0`, prompt système forçant une transcription mot-à-mot dans la langue d'origine (pas de traduction, pas de résumé). Fichier wav temporaire supprimé dans un `finally`.

Pourquoi wav 16kHz mono : format standard d'entraînement des modèles STT (Whisper et la plupart en 16kHz mono), réduit la taille du payload base64 (coût/latence) sans perte de qualité perceptible sur de la voix. **Pourquoi temperature=0** : déterminisme — même transcription à chaque appel sur le même audio, aucune créativité/hallucination souhaitée pour une tâche factuelle.

Évolution via l'historique : modèle d'abord *hardcodé*, puis rendu configurable par variable d'env ; la température a été ajoutée en tout dernier commit (passant d'un commentaire `#ajouter la temperature` à une vraie valeur).

backend/llm.py — le "classifier" (en réalité un module de structuration)

`generate_report(transcript)` → prompt système chargé depuis `agent_context.txt`, `response_format=json_object`, `temperature=0`, `max_tokens=4096`, streaming activé, puis reconstruction + parse JSON.

Le prompt impose une structure stricte : `summary`, `speaker_count`, `speakers`, `themes`, `actions` (`id/label/assignee`), `transcript` (segments `speaker/time/text`). Règles anti-hallucination explicites : ne jamais inventer un nom absent du texte (utiliser "Inconnu"/"Interlocuteur N"), répondre dans la langue d'entrée.

Ce n'est pas une classification au sens strict (catégorisation) mais une extraction structurée — le terme "classifier" dans les commits est un abus de langage historique.

Historique du modèle (commit `0d10ab2` "modification classifier → Qwen3.7-Max") : le module tournait initialement sur **Groq** avec `openai/gpt-oss-20b` (modèle open-weight OpenAI hébergé par Groq, `reasoning_effort="medium"`), remplacé ensuite par **Together AI / Qwen3.7-Max** en streaming. Le même commit supprime aussi un dossier de *handoff design* sans rapport (nettoyage de repo).

backend/main.py

`normalize_url(url)` → préfixe `https://` si `FRONTEND_URL` n'a pas de schéma, évite un CORS cassé.

POST /recordings(audio) → sauvegarde temporaire → `callapi()` (Voxtral) → `generate_report()` (Qwen) → suppression du fichier temporaire → renvoie `{status, Compte-rendu, transcription}`. Pas d'écriture DB sur cette branche.

POST /recap → endpoint stub non implémenté ("not yet able to transcript it").

Fragile : aucune gestion d'erreur explicite autour des appels API dans `main.py` (pas de `try/except`) ; le fichier temporaire d'origine (avant conversion wav) n'est supprimé qu'en fin de fonction réussie, pas dans un `finally` — reste sur disque en cas d'exception intermédiaire.

.env.example — inventaire des providers testés

Contient `MEETING_BAAS_API_KEY`, `GLADIA_API_KEY`, `VEXA_API_KEY`, `GROQ_API_KEY` (résidu de l'ancienne implé), `MISTRAL_API_KEY`, `MISTRAL_MODEL`, `TOGETHER_API_KEY`, `TOGETHER_MODEL=Qwen/Qwen3.7-Max`, `DATABASE_URL` — confirme que plusieurs providers STT ont été centralisés/testés à travers les branches.

Pipeline dictaphone complet

1. Frontend envoie un fichier audio (multipart/form-data) → **POST /recordings**
2. `main.py` sauvegarde le fichier dans temp/
3. `stt.py::callapi()` → conversion wav 16kHz mono (ffmpeg) → Mistral/Voxtral (`temperature=0`)
 - transcription brute fidèle, langue d'origine préservée
4. `llm.py::generate_report()` → transcription + prompt système → Qwen (Together, streaming, JSON forcé)
 - compte-rendu structuré (résumé, intervenants, thèmes, actions, transcript diarisé)
5. Fichier temporaire supprimé, réponse JSON renvoyée au frontend
6. (Pas de sauvegarde DB sur cette branche — branché plus tard, voir section 4)

Contexte général Voxtral (voir section 6 pour les chiffres sourcés)

Voxtral est le modèle audio de Mistral AI (sorti mi-2025, déclinaisons Small/Mini), basé sur l'architecture Mistral avec un encodeur audio ajouté. Positionné comme alternative à Whisper avec : compréhension audio "native" au-delà de la transcription pure, support multilingue avec détection automatique de la langue, et surtout — expliquant le choix d'architecture ici — un modèle "chat" (`client.chat.complete`) permettant d'injecter des instructions directement dans le prompt ("ne traduis jamais", "réponds uniquement avec la transcription brute") en un seul appel, plutôt qu'un pipeline transcription + reformattage séparé.

Questions probables du jury — pipeline dictaphone

Q. Pourquoi convertir en wav 16kHz mono avant d'envoyer à Voxtral ?

R. Format standard des modèles de reconnaissance vocale, réduit la taille du payload base64 (coût/latence), sans perte de qualité perceptible pour de la voix.

Q. Pourquoi température à 0 dans `stt.py` et `llm.py` ?

R. Déterminisme : même transcription/même compte-rendu à chaque appel sur le même audio, sans variation créative — critique pour une tâche d'extraction factuelle.

Q. Pourquoi Voxtral plutôt que Whisper ?

R. Choix pragmatique : déjà dans l'écosystème Mistral utilisé ailleurs dans le projet, API simple, capacité multimodale native, et — chiffres à l'appui (section 6) — meilleur WER que Whisper large-v3 sur les benchmarks officiels Mistral (FLEURS, Common Voice) pour un coût par minute inférieur.

Q. Pourquoi Qwen pour le "classifier" et pas GPT/Claude ?

R. Le repo est passé de Groq (gpt-oss-20b) à Together AI (Qwen3.7-Max) — arbitrage coût/latence via un provider d'inférence mutualisée bon marché pour un usage étudiant, avec un JSON mode fiable.

Q. Que fait exactement le "classifier" ?

R. Ce n'est pas une classification mais une extraction structurée (JSON) : résumé, intervenants, thèmes, actions assignées, transcript diarisé — piloté entièrement par le prompt système agent_context.txt.

Q. Comment gérez-vous les erreurs si Mistral/Together timeout ?

R. Honnêtement : aucune gestion d'erreur explicite actuellement (pas de try/except autour des appels API) — axe d'amélioration identifié.

Q. Pourquoi deux définitions de convert_to_wav() dans stt.py ?

R. Doublet de code, résidu d'itération (la seconde écrase la première à l'exécution) — bug de code review à reconnaître, pas un choix voulu.

Q. Pourquoi ne pas sauvegarder en base sur cette branche ?

R. Le focus de cette branche était de prouver le pipeline STT+report en isolation avant de le brancher à la persistance, faite sur feature/link_dictaphone_to_database.

4. Liaison dictaphone ↔ base de données

feature/link_dictaphone_to_database

```
3912af6 ajout de la liaison du dictaphone à la base de donnée via sqlalchemy
3009cb2 passer user_id en nullable pour pouvoir tester la feature
e8dcb3f fix user_id error to int (en réalité modifie "id")
05a221a fix user_id error to int (modifie "user_id")
```

backend/db.py (nouveau)

DATABASE_URL / engine / SessionLocal / Base → point d'entrée SQLAlchemy : engine Postgres (Railway), factory de sessions (autoflush=False), classe déclarative de base.

Pourquoi autoflush=False : évite un flush automatique avant chaque requête, donne un contrôle explicite sur le moment du commit/flush.

backend/models.py — trajectoire du bug id/user_id

Étape	État	Pourquoi
Initial (3912af6)	id et user_id en UUID(as_uuid=True)	Choix par défaut "sécurisé" classique pour des identifiants publics
3009cb2	user_id requis → Form(None) + nullable=True	Débloquer les tests end-to-end : le frontend n'envoie pas encore d'auth réelle
e8dcb3f	id : UUID → Integer	La table users (gérée côté auth, autre branche) utilise des ID entiers auto-incrémentés
05a221a	user_id : UUID → Integer	Aligner le type de la FK potentielle avec le schéma users (incompatibilité de type à l'insertion)

Fragile : aucune vraie ForeignKey("users.id") déclarée — user_id est un Integer nu, sans contrainte référentielle ni relationship ORM. L'intégrité référentielle n'est pas garantie par la DB.

backend/main.py — route POST /recordings

get_db() → générateur FastAPI standard (yield + finally: db.close()), injecté via Depends — garantit la fermeture de session même en cas d'exception.

records(audio, user_id, db) → upload → temp → callapi() (Voxtral) → generate_report() (Qwen) → suppression fichier temp → une seule transaction add/commit/refresh qui persiste transcription ET reporting.

Fragile : callapi (Mistral) et generate_report (Together) sont synchrones/bloquants, appelés dans un handler async def — bloque l'évent loop FastAPI pendant tout l'appel réseau (pas de run_in_threadpool). user_id: str = Form(None) typé str côté API alors que la colonne DB est Integer — incohérence de typage entre couche API et couche DB. Aucun try/except avec rollback() explicite autour du commit.

Schéma final (HEAD de la branche)

Colonne	Type	Note
id	Integer, PK	Autoincrement implicite Postgres
user_id	Integer, nullable	Pas de FK déclarée vers users.id
name	String, not null	Nom du fichier audio uploadé
source	String, not null	"dictaphone" "meeting_bot"
created_at	String, not null	ISO-8601 UTC généré en Python
transcription	String, not null	Texte brut Voxtral
reporting	JSONB, not null	Sortie structurée du LLM

Questions probables du jury

Q. Pourquoi user_id était nullable ?

R. Pour pouvoir tester le flux audio→DB avant que l'authentification/l'envoi du user_id depuis le frontend soit finalisée — contrainte temporaire de développement, pas un choix définitif.

Q. Quel était le bug de type sur user_id/id ?

R. id et user_id étaient en UUID alors que la table users utilise des entiers auto-incrémentés ; insérer un user_id réel provoquait une erreur de type Postgres. Les deux colonnes ont été repassées en Integer.

Q. Pourquoi SQLAlchemy plutôt qu'un client SQL brut ?

R. ORM = mapping objet-relationnel, évite d'écrire du SQL à la main, protège nativement contre l'injection SQL (requêtes paramétrées), s'intègre nativement avec Depends de FastAPI pour la session par requête.

Q. Comment est gérée la session DB ?

R. Une session par requête HTTP via get_db() injecté en dépendance : yield puis close() en finally. Commit explicite après add(). Point faible assumé : pas de rollback explicite en cas d'échec du commit.

Q. Il n'y a pas de ForeignKey vers users, pourquoi ?

R. Pas encore implémenté à ce stade — la feature d'auth/users était développée en parallèle sur une autre branche. À corriger avant mise en prod pour garantir l'intégrité des données.

5. Vexa vs Meeting BaaS & connexion calendrier automatique

Branches : `lab/test_vexa_meeting_baas`, `origin/lab/benchmark_meeting_bas`, `origin/features/join-auto-bot-calendars-api`.

5.1 — Vexa vs Meeting BaaS (lab/test_vexa_meeting_baas)

Branche laboratoire où les deux approches coexistent (`vexa_bot.py` et `meetingbaas_client.py`) avant que le projet ne tranche. Un `README.md` de cette branche documente explicitement : *"vexa_bot.py est un chemin alternatif qui envoie le bot et affiche le transcript dans le terminal — mais ne pousse pas encore en base. Pas branché sur save_meeting.py."*

	Vexa (abandonné)	Meeting BaaS (retenu)
API	<code>api.cloud.vexa.ai</code>	<code>api.meetingbaas.com/v2</code>
Détection réunion	Regex manuelle sur URL Meet / texte d'invitation Teams (Conference ID + Passcode)	URL de réunion directe
Transcription	<code>transcription_tier</code> : "realtime", polling	Provider configurable (Gladia/Deepgram testés)
Statut	Jamais branché à la persistance DB	Pipeline complet en production (section 2)

Aucune preuve de désavantage technique chiffré documentée dans le code contre Vexa — c'est un choix de scope (finir un seul pipeline proprement) plutôt qu'un rejet motivé par un benchmark. Voir section 6 pour la comparaison tarifaire, où Vexa reste compétitif (\$0,50/h tout compris ou self-hosted gratuit).

5.2 — Benchmark Meeting BaaS (commit dba3428 "essaie pour benchmark de meeting_baas")

Test exploratoire isolé (scratch de dev), deux classes :

SpeechToTextAgent → `POST /v2/bots` avec `transcription_config.provider="deepgram"` (donc pas Gladia ici — **preuve que plusieurs providers de transcription ont été essayés via Meeting BaaS**, Gladia et Deepgram) + `custom_params.summarization` côté Meeting BaaS. `_wait_for_transcript` gère plusieurs formats de réponse possibles (extraction défensive).

TextClassifierAgent → appelle Groq (qwen/qwen3-32b, JSON mode) pour classifier le compte-rendu (catégorie, résumé court, actions clés, sentiment).

Ce brouillon a mené au choix Groq/Qwen initial, avant que la branche finale `visio_to_database` bascule sur Together AI/Qwen3.7-Max (probablement pour la stabilité/limites de rate de Groq en usage réel — non documenté explicitement, à confirmer si le prof insiste).

5.3 — Connexion calendrier automatique (features/join-auto-bot-calendars-api)

Module backend/`calendar_bots/` (router FastAPI séparé) :

oauth.py :: build_authorize_url() → flow OAuth Google "bring-your-own-credentials" (c'est le backend qui gère l'échange OAuth, Meeting BaaS ne reçoit que le `refresh_token` final).

Force `access_type=offline&prompt=consent` (sinon Google ne renvoie un `refresh_token` qu'à la toute première autorisation).

`oauth.py :: exchange_code_for_refresh_token()` → échange le code contre le `refresh_token`, message d'erreur explicite si absent (cas déjà-autorisé → révoquer sur `myaccount.google.com`).

`client.py :: register_calendar()` → `POST /calendars` (envoie le `refresh_token` à Meeting BaaS), gère l'idempotence via le code d'erreur `FST_ERR_CALEDAR_CONNECTION_ALREADY_EXISTS`.

`client.py :: get_event() / schedule_bot()` → `POST /calendars/{id}/bots`, `recording_mode="speaker_view"`.

`rules.py :: should_join(event)` → règle métier : le bot rejoint SEULEMENT si son adresse (`BOT_ATTENDEE_EMAIL`) figure dans les attendees de l'événement Google Calendar.

`store.py` → persistance JSON fichier local (`calendar_store.json`), volontairement temporaire/mono-utilisateur.

`main.py` → 3 routes : `GET /oauth/authorize` (redirige vers Google, state CSRF en mémoire), `GET /oauth/callback` (vérifie state, échange code, enregistre calendrier), `POST /webhook` (reçoit `calendar.event_created/updated`, vérifie la signature via `svix`, boucle sur les instances, `should_join`, programme le bot, déduplique).

Questions probables du jury

Q. Pourquoi Meeting BaaS plutôt que Vexa ou un bot fait maison ?

R. Vexa a été testé en parallèle mais jamais branché à la persistance. Meeting BaaS gère nativement multi-plateformes, plusieurs providers STT interchangeables, la salle d'attente, et une API Calendar clé en main — moins de code à maintenir qu'un bot maison (Selenium/Playwright dans une visio, très fragile).

Q. Comment fonctionne l'auto-scheduling via calendrier ?

R. OAuth Google → `refresh_token` transmis à Meeting BaaS qui s'abonne aux changements du calendrier → `webhook calendar.event_created/updated` reçu → pour chaque instance, vérification `should_join` → si oui, `POST /calendars/{id}/bots` programme automatiquement l'envoi du bot à l'heure de la réunion.

Q. Comment est sécurisé le webhook Meeting BaaS ?

R. Vérification de signature HMAC via la librairie `svix`, secret partagé en variable d'env. Le endpoint OAuth callback est protégé par un state CSRF à usage unique.

Q. Pourquoi gérer l'OAuth nous-mêmes plutôt que de laisser Meeting BaaS le faire ?

R. Modèle bring-your-own-credentials : on garde le contrôle du client OAuth Google, Meeting BaaS ne reçoit que le `refresh_token` final — moins de dépendance/confiance à accorder à un tiers sur nos

identifiants Google.

Q. Que se passe-t-il si un événement webhook arrive deux fois ?

R. `store.is_event_scheduled(event_id)` déduplique côté serveur avant de reprogrammer un bot.

Q. Pourquoi un stockage JSON local plutôt qu'une vraie table en base ?

R. Choix pragmatique temporaire, assumé dans le code : pas encore de notion de compte/session multi-utilisateur à ce stade — à remplacer dès que l'auth sera en place.

Q. Quels providers de transcription ont été testés via Meeting BaaS ?

R. Gladia (retenu par défaut) et Deepgram (testé dans le benchmark dba3428) via `transcription_config.provider` — arbitrage qualité/coût, chiffres précis en section 6.

Q. Si l'utilisateur révoque l'accès Google après connexion, que se passe-t-il ?

R. Non géré explicitement (pas de détection de `refresh_token` invalide) — limite connue à mentionner honnêtement plutôt qu'un flow réellement traité.

6. Historique des benchmarks STT — pourquoi Voxtral au final

Branches : `lab/benchmark-STT`, `feature/speech_to_text_agent`, `feature/dictaphone (v1)`. Reconstitution factuelle basée sur `git log/diff` — les points non documentés dans le code sont signalés comme tels plutôt qu'inventés.

Chronologie

Étape	Ce qui a été testé
1. <code>feature/dictaphone</code> (commit 54a6149, version initiale)	<code>stt.py</code> utilisait Groq API avec whisper-large-v3 (API gratuite/rapide de Groq, pas de Whisper local). Choix simple, orienté rapidité de mise en œuvre.
2. <code>feature/speech_to_text_agent</code>	Branche jamais développée — seul le commit initial vide existe. Abandonnée au profit de <code>benchmark-STT</code> .
3. <code>lab/benchmark-STT</code> (commits "vibecoding")	Vrai benchmark multi-provider : 6 providers testés en parallèle via <code>asyncio.gather</code> — Groq whisper-large-v3, Groq whisper-large-v3-turbo, Deepgram nova-2, AssemblyAI best, Speechmatics enhanced, OpenAI whisper-1. Chaque résultat croisé avec 4 LLM classifieurs (Groq llama-3.3-70b, llama-3.1-8b-instant, mixtral-8x7b, gemma2-9b-it), résultats persistés avec durée mesurée par combo.
4. <code>feature/dashboard</code> (commit 675c01a)	Passage direct de Groq whisper-large-v3 à Mistral voxtral-small-latest , sans étape de benchmark comparatif visible dans le code spécifiquement pour Voxtral.

⚠ **Point important à anticiper** : Voxtral/Mistral n'apparaît dans AUCUN des 6 providers testés sur `lab/benchmark-STT`. Le switch vers Voxtral arrive après, sans étape de comparaison chiffrée documentée dans le code pour ce modèle précis. Réponse honnête recommandée : *"Le benchmark initial comparait des providers généralistes (Whisper via différents hébergeurs, Deepgram, AssemblyAI, Speechmatics). Voxtral est arrivé après, comme choix pragmatique : un seul provider (Mistral) sert à la fois le LLM et le STT, ce qui simplifie la stack (une seule clé API, un seul SDK) plutôt que de multiplier les fournisseurs — et les benchmarks officiels Mistral (section 7) confirment a posteriori que ce choix n'est pas au détriment de la qualité."*

Critères observés dans le code du benchmark

- **Latence** — `duration_ms` mesuré et persisté pour chaque appel
- **Disponibilité de la clé** (donc coût d'accès implicite) — seuls les providers dont la clé API est présente en env sont testés
- **Qualité** — aucune métrique automatisée (pas de WER calculé) : la comparaison qualité semble avoir été faite à l'œil, en lisant les transcriptions stockés

Questions probables du jury

- Q. Quelles solutions STT avez-vous comparées ?**
- R.** Groq Whisper-large-v3 / large-v3-turbo, Deepgram Nova-2, AssemblyAI, Speechmatics, OpenAI Whisper-1 — benchmarkées en parallèle avec mesure de latence et double passage par 4 LLM de résumé.

Q. Pourquoi avoir écarté Deepgram/AssemblyAI/Speechmatics/OpenAI ?

R. Non documenté par une métrique automatisée dans le code (pas de fichier de résultats consultable). Réponse honnête : le benchmark mesurait la latence automatiquement, le choix final s'est fait sur lecture qualitative des transcripts — limite reconnue, un scoring qualité type WER serait la suite logique.

Q. Pourquoi Voxtral alors qu'il n'était pas dans le benchmark initial ?

R. Choix pragmatique arrivé après coup : mutualiser le provider Mistral pour le STT et le LLM simplifie la stack (une seule clé API). Les benchmarks officiels Mistral publiés depuis confirment que Voxtral bat Whisper large-v3 sur WER (section 7).

Q. Aviez-vous testé Whisper en self-hosted/local ?

R. Non — uniquement des API hébergées (Groq héberge whisper-large-v3, ce n'est pas un déploiement local).

Q. Le coût a-t-il été un critère de sélection ?

R. Groq est gratuit (mentionné en commentaire ".env.example"), ce qui explique le choix initial en V1. Pour les autres providers testés, pas de comparaison de coût persistée dans le code — seule la latence est mesurée automatiquement.

7. Benchmarks & tarifs sourcés

Sources officielles vérifiées : mistral.ai/news/voxtral, rapport technique [arXiv:2507.13264](https://arxiv.org/abs/2507.13264), mistral.ai/pricing/api, meetingbaas.com/api-pricing, gladia.io/pricing, vexa.ai/pricing. Les valeurs de WER ont été lues directement sur les graphiques officiels du blog Mistral (annonce du 15 juillet 2025) — précision $\pm 0,1-0,2$ point, à préciser à l'oral.

7.1 — Word Error Rate (WER, plus bas = meilleur)

Benchmark	Voxtral Small	Voxtral Mini Transcribe	Voxtral Mini	Scribe (ElevenLabs)	GPT-4o mini Transcribe	Gemini 2.5 Flash	Whisper large-v3
English short-form	6,3%	6,5%	6,8%	6,4%	8,9%	8,1%	7,7%
English long-form	11,1%	10,9%	10,7%	8,3%	11,2%	9,5%	11,5%
Common Voice 15.1	5,7%	6,4%	7,9%	5,9%	12,3%	8,8%	14,1%
FLEURS (moy. multilingue)	5,1%	5,6%	7,1%	4,9%	5,7%	7,0%	8,3%

À retenir : Voxtral Small bat systématiquement Whisper large-v3 sur les 4 benchmarks agrégés ET sur les 9 langues FLEURS testées individuellement (revendication Mistral vérifiée). **Nuance honnête :** face à Scribe (ElevenLabs), Voxtral ne gagne que sur 2 des 4 benchmarks (English short-form, Common Voice) — Scribe reste meilleur en long-form et en moyenne FLEURS, et nettement meilleur sur italien/hindi/arabe. Le marketing Mistral sélectionne ses comparaisons — bon réflexe à montrer au jury en le disant soi-même.

7.2 — Rapport prix/performance (graphique officiel FLEURS)

Modèle	WER FLEURS	Prix estimé /min
Voxtral Mini	7,1%	~\$0,001
Voxtral Mini Transcribe	5,55%	~\$0,002
Voxtral Small (utilisé dans le projet)	5,1%	~\$0,004
GPT-4o mini Transcribe	5,7%	~\$0,003
Gemini 2.5 Flash	7,05%	~\$0,0027
Whisper large-v3	8,3%	~\$0,006
Scribe (ElevenLabs)	4,9%	~\$0,010

7.3 — Prix officiel API Voxtral (mistral.ai/pricing/api)

Modèle	Prix
--------	------

Voxtral Small (voxtral-small-latest, utilisé)	Audio input : \$0,004/min (≈ \$0,24/h) • Texte input : \$0,10/M tokens • Output : \$0,40/M tokens
Voxtral Mini Transcribe 2	\$0,003/min
Voxtral Mini Transcribe Realtime	\$0,006/min

Le chiffre "\$0,001/min" parfois cité vient du blog d'annonce initial et correspond à Voxtral **Mini**, pas Small (le modèle utilisé dans ce projet). Retenir **\$0,004/min = \$0,24/h** pour `voxtral-small-latest`.

7.4 — Prix Meeting BaaS (meetingbaas.com/api-pricing)

Modèle : abonnement mensuel (quotas) + jetons à l'usage (n'expirent jamais).

Abonnement	Prix/mois	Quota bots/jour
Pay-as-you-go	Gratuit	75
Pro	\$99	300
Scale	\$199	1000 (-5% tokens)
Enterprise	\$299	3000 (-10% tokens)

Pack de tokens	Prix	Prix/token
Boost	\$50 → 100 tokens	\$0,50
Growth	\$100 → 220 tokens	\$0,45
Power	\$625 → 1550 tokens	\$0,40
Infinity	\$1500 → 4250 tokens	\$0,35 (meilleur tarif)

Consommation : enregistrement brut = 1,00 token/h ; + transcription Gladia (provider par défaut du projet) = +0,25 token/h → **1,25 token/h au total**. Coût effectif enregistrement + transcription : **entre \$0,4375/h** (meilleur tarif) **et \$0,625/h** (tarif de base), plus l'abonnement mensuel.

7.5 — Prix Gladia seul, pour référence (gladia.io/pricing)

Offre	Prix
Starter (pay-as-you-go)	\$0,61/h (async) / \$0,75/h (temps réel)
Growth (volume engagé)	jusqu'à \$0,20/h (async) / \$0,25/h (temps réel)
Enterprise	Sur devis

Meeting BaaS + Gladia intégré revient à \$0,44-0,625/h, moins cher que le tarif public Gladia seul (\$0,61-0,75/h) — suggère que Meeting BaaS négocie un tarif Gladia réduit en marque blanche.

7.6 — Vexa, pour comparaison (vexa.ai/pricing)

Offre	Prix
-------	------

Individuel (cloud managé)	\$12/mois, 1 bot simultané, réunions illimitées
Pay-as-you-go	\$0,30/h infra + \$0,20/h transcription = \$0,50/h tout compris
Self-hosted (Apache 2.0)	Gratuit, infra à sa charge

7.7 — Synthèse comparative

	Coût transcription	Coût bot réunion + transcription
Voxtral Small (dictaphone)	\$0,004/min = \$0,24/h	n/a (pas de bot)
Gladia seul	\$0,61-0,75/h (ou \$0,20-0,25/h en volume)	n/a
Meeting BaaS + Gladia (retenu)	inclus	\$0,4375-0,625/h + abonnement \$0-299/mois
Vexa (cloud)	inclus	\$0,50/h, pas d'abonnement obligatoire
Vexa (self-hosted)	gratuit	gratuit (infra à sa charge)

Limites de cette recherche à mentionner si challengé : WER lus visuellement sur des graphiques (±0,1-0,2 pt) faute de tables chiffrées accessibles en texte ; pas de comparaison officielle Voxtral vs Deepgram Nova trouvée avec chiffres précis ; prix Meeting BaaS/Gladia/Vexa datés de la recherche (juillet 2026), à revérifier si la soutenance a lieu plus tard.

8. FAQ transversale — questions probables du jury

Sur l'architecture générale

Q. C'est quoi le schéma de données global du projet ?

R. Une table unique recaps (SQLAlchemy/Postgres) avec un champ discriminant source ("dictaphone" ou "visio"), stockant transcription (texte brut) et reporting (JSONB structuré par le LLM). Pas encore de vraie relation FK vers une table users — l'authentification est un chantier séparé en cours.

Q. Pourquoi une seule table plutôt qu'une table par type de source ?

R. Le résultat final (transcription + compte-rendu) a la même forme quelle que soit l'origine — dupliquer le schéma aurait ajouté de la complexité sans bénéfice, le champ source suffit à distinguer/filtrer.

Q. Pourquoi FastAPI plutôt que Flask/Django ?

R. Async natif (utile pour les I/O réseau vers Mistral/Together/Meeting BaaS), typage Pydantic intégré, performance, écosystème moderne standard pour microservices Python.

Q. Pourquoi deux LLM différents ont été testés (Groq/Qwen3-32b puis Together/Qwen3.7-Max) ?

R. Le benchmark exploratoire a d'abord testé Groq pour sa vitesse d'inférence gratuite ; la version finale utilise Together AI avec un modèle plus large — arbitrage vitesse/coût de dev rapide vs qualité de résumé en usage réel.

Sur les coûts

Q. Combien coûte une minute de transcription dictaphone (Voxtral) ?

R. \$0,004/minute pour voxtral-small-latest (source : mistral.ai/pricing/api), soit environ \$0,24/heure d'audio.

Q. Combien coûte une réunion visio enregistrée + transcrite via Meeting BaaS ?

R. Entre \$0,44 et \$0,625 par heure de réunion selon le palier tarifaire de tokens, plus un abonnement mensuel optionnel (\$0 à \$299/mois selon le volume de bots nécessaire par jour).

Q. Est-ce que Voxtral coûte plus ou moins cher que Whisper ?

R. Moins cher : \$0,004/min pour Voxtral Small contre environ \$0,006/min pour Whisper large-v3 (estimation du graphique officiel Mistral), pour un WER meilleur sur la plupart des benchmarks.

Sur la sécurité / robustesse

Q. Les clés API sont-elles bien protégées ?

R. Oui, centralisées via config.py avec lecture depuis variables d'environnement uniquement, jamais hardcodées, avec échec explicite si absentes (fail-fast).

Q. Que se passe-t-il en cas de panne d'un service tiers (Meeting BaaS, Mistral, Together) ?

R. Réponse honnête, variable selon la branche : côté visio, un échec fait retirer le bot proprement (remove_bot) et logue l'erreur ; côté dictaphone, la gestion d'erreur est plus faible (pas de try/except explicite autour des appels réseau) — axe d'amélioration reconnu.

Sur les choix de stack

Q. Pourquoi Postgres et pas un autre SGBD ?

R. Support natif du type JSONB (essentiel pour stocker le reporting structuré sans schéma rigide), largement utilisé en production, bien intégré avec SQLAlchemy et Railway (hébergement utilisé).

Q. Pourquoi stocker le compte-rendu en JSON plutôt qu'en colonnes normalisées ?

R. Le format généré par le LLM évolue à chaque itération de prompt (nouveaux champs, structure affinée) — le JSONB évite une migration de schéma SQL à chaque changement, au prix de requêtes SQL moins directes sur le contenu.

9. Fiche "points faibles à assumer honnêtement"

Un jury apprécie généralement plus un étudiant qui reconnaît et argumente ses angles morts qu'un étudiant qui prétend que tout est parfait. Voici la liste des faiblesses réelles identifiées dans le code, classées par sévérité, avec une reformulation honnête prête à l'oral.

[Majeur] Pas de user_id sur le pipeline visio — le dernier commit de feature/visio_to_database a retiré user_id du modèle Recap, alors que feature/link_dictaphone_to_database l'a justement ajouté côté dictaphone. Incohérence entre branches à assumer : *"L'authentification n'est pas encore branchée sur le flux visio, c'est le prochain chantier avant de fusionner les deux branches."*

[Majeur] Aucune ForeignKey vers une table users — user_id est un Integer nu sans contrainte référentielle ni relation ORM sur aucune des deux branches. *"L'intégrité référentielle sera ajoutée une fois le module d'authentification finalisé."*

[Moyen] Gestion d'erreur incomplète — pas de try/except autour des appels réseau (Mistral, Together) dans le pipeline dictaphone ; pas de validation Pydantic de la sortie JSON du LLM. *"Le MVP privilégiait la démonstration du flux complet ; le durcissement des erreurs réseau est identifié comme prochaine étape."*

[Moyen] Polling bloquant et synchrone — wait_for_completion utilise requests (bloquant) dans une BackgroundTask FastAPI jusqu'à 1h, ne scale pas avec plusieurs réunions simultanées. *"Une architecture event-driven avec les webhooks Meeting BaaS (déjà utilisés côté calendrier) remplacerait avantageusement ce polling."*

[Moyen] BackgroundTasks non persistant — si le serveur redémarre pendant l'attente d'un bot, la tâche est perdue silencieusement (pas de Celery/RQ). *"Acceptable pour une démo, une vraie queue de tâches serait nécessaire en production."*

[Mineur] Code mort — vexa_bot.py — jamais appelé depuis le pivot vers Meeting BaaS, gardé comme trace du choix technique. *"Volontairement conservé pour la traçabilité de la démarche de comparaison, à nettoyer avant un merge en prod."*

[Mineur] Double convert_to_wav() dans stt.py — deux définitions de la même fonction, la seconde écrase la première (dead code). *"Résidu d'itération rapide, sans impact fonctionnel mais à nettoyer."*

[Mineur] created_at en String plutôt que DateTime — perte du tri natif SQL et des fonctions temporelles Postgres. *"Choix pour simplifier la sérialisation JSON côté frontend sans conversion de timezone ; une vraie colonne DateTime serait préférable à terme."*

[Mineur] Incohérence de typage user_id (str côté API, Integer côté DB) — *"Fonctionne car SQLAlchemy/psycopg convertit les strings numériques implicitement, mais un typage Pydantic explicite serait plus propre."*

[Mineur] Choix Voxtral non issu du benchmark multi-provider initial — le benchmark lab/benchmark-STT ne testait pas Voxtral. *"Choix pragmatique de mutualisation de provider (Mistral pour STT et LLM), confirmé a posteriori par les benchmarks officiels Mistral qui montrent un WER meilleur que Whisper large-v3."*

[Mineur] Endpoint POST /recap non implémenté (stub) — présent dans main.py mais renvoie un message "not yet able to transcript it". *"Endpoint prévu pour une future fonctionnalité de re-traitement, non prioritaire pour la démo actuelle."*

1. Vue d'ensemble de l'architecture

Le projet **Scribe** transforme des réunions (audio dictaphone ou visioconférence) en comptes-rendus structurés. Deux pipelines distincts convergent vers la **même table de base de données** :

```
PIPELINE A – Dictaphone (feature/dashboard, link_dictaphone_to_database)
Frontend upload audio → FastAPI /recordings → conversion ffmpeg (wav 16kHz mono)
→ Mistral Voxtral (STT, transcription directe) → Together AI / Qwen (compte-rendu JSON)
→ SQLAlchemy INSERT (source="dictaphone") → Postgres

PIPELINE B – Visioconférence (feature/visio_to_database)
Frontend → FastAPI POST /bots {meeting_url} → Meeting BaaS (bot rejoint Teams/Meet/Zoom,
enregistre, transcrit via Gladia en interne – le backend ne fait AUCUN traitement audio)
→ polling wait_for_completion() → transcript.py (parsing diarisation)
→ Together AI / Qwen (compte-rendu JSON) → SQLAlchemy INSERT (source="visio") → Postgres

PIPELINE C – Auto-scheduling (annexe) (features/join-auto-bot-calendars-api)
OAuth Google Calendar → refresh_token transmis à Meeting BaaS → webhook calendar.event_*
→ règle should_join() (bot invité ?) → programmation automatique du bot au bon horaire
```

Point clé à savoir dire en une phrase

Pourquoi : le backend ne fait jamais de traitement audio lui-même sur le pipeline visio — Meeting BaaS gère le join multi-plateforme + l'enregistrement + la transcription (via Gladia). Le backend, lui, ne s'occupe que du *post-traitement* : parsing, structuration LLM, persistance. Sur le pipeline dictaphone en revanche, c'est le backend qui appelle directement Voxtral pour la transcription, car il reçoit un fichier audio brut sans intermédiaire.

Schéma de données (table unique recaps)

Colonne	Type	Rôle
recap_id / id	Integer, PK	Identifiant auto-incrémenté (a été migré depuis UUID, cf. section 4)
name	String, not null	Nom du fichier audio (dictaphone) ou liste des participants (visio)
source	String, not null	Discriminant : "dictaphone" ou "visio" — permet de réutiliser une seule table pour les deux pipelines
created_at	String (ISO-8601 UTC)	Date de création, formatée en Python plutôt qu'en vrai DateTime SQL
transcription	String, not null	Texte brut de la transcription (Voxtral ou Gladia selon la source)

reporting	JSONB, not null	Compte-rendu structuré généré par le LLM : {summary, themes, actions, speakers}
user_id	Integer, nullable	Présent côté dictaphone (feature/link_dictaphone_to_database) — absent côté visio depuis le dernier commit (voir section 2, point faible n°6)
emails	JSONB, nullable	Emails des participants (visio), en attente d'un vrai système d'auth

Pas de ForeignKey vers une table users à ce stade — l'authentification est un chantier séparé, non encore branché sur ces deux pipelines.

Services externes utilisés et leur rôle exact

Service	Rôle dans le projet	Utilisé par
Meeting BaaS	Fait rejoindre un bot à une réunion Teams/Meet/Zoom, l'enregistre, orchestre la transcription via un provider configurable	Pipeline visio
Gladia	Provider de transcription + diarisation, appelé <i>par Meeting BaaS en interne</i> (provider par défaut, transcription_config.provider)	Pipeline visio (indirect)
Mistral AI (Voxtral)	Modèle audio appelé directement par le backend pour transcrire un fichier uploadé	Pipeline dictaphone
Together AI (Qwen)	LLM appelé pour structurer la transcription brute en compte-rendu JSON (résumé, thèmes, actions)	Les deux pipelines (llm.py)
Google Calendar (OAuth)	Détection automatique des réunions où le bot est invité	Pipeline C (auto-scheduling)
Vexa	Alternative open-source à Meeting BaaS, testée puis abandonnée (code résiduel vexa_bot.py)	Aucun (code mort, gardé pour traçabilité)